Метод Монте-Карло

1. Что такое метод Монте-Карло?

Представьте, что вам нужно узнать, сколько конфет в большой банке, не открывая её. Можно спросить у друзей, пусть каждый наугад назовёт число, а потом взять среднее. Чем больше друзей вы спросите, тем ближе будет ответ к истине. Примерно так же работает метод Монте-Карло – мы проводим много случайных экспериментов и по их результатам делаем выводы о том, что нас интересует.

Название методу дали в честь знаменитого казино в Монте-Карло, потому что главный инструмент здесь – случайность, как в рулетке или в игральных костях.

Где применяется?

- В физике (расчёт поведения частиц),
- в финансах (прогноз цен на акции),
- в компьютерных играх (генерация миров),
- в науке (оценка сложных интегралов) и даже в искусственном интеллекте.

Главная идея:

Если мы не можем точно вычислить величину (например, площадь фигуры сложной формы), мы можем многократно «бросать» случайные точки в некоторую область и смотреть, сколько из них попало туда, куда нужно. Доля попаданий примерно равна отношению искомой площади к площади всей области.

2. Классический пример: вычисление числа π

Число π (пи) – это отношение длины окружности к её диаметру. Приблизительно $\pi \approx 3{,}14159$. Мы попробуем оценить π с помощью случайных бросаний точек.

Как это устроено:

- Возьмём квадрат со стороной 2 (от –1 до 1 по оси x и по оси y). Площадь такого квадрата равна 4.
- Впишем в этот квадрат круг радиусом 1 с центром в начале координат.

Площадь круга равна $\pi \cdot 1^2 = \pi$.

- Если мы будем бросать точки случайным образом внутрь квадрата, то вероятность того, что точка окажется внутри круга, равна отношению площади круга к площади квадрата:

$P = \pi / 4$.

- Следовательно, π можно оценить так:

$\pi \approx 4 \cdot$ (число точек внутри круга) / (общее число точек).

Чем больше точек мы бросим, тем точнее будет приближение.

3. Моделирование в Excel

Мы сделаем простую таблицу, которая будет генерировать случайные координаты точек, проверять, попала ли точка в круг, и считать приближённое значение π. Также построим график, чтобы увидеть наши точки.

Ход работы:

1. Откройте новый лист в Excel.
2. Создайте заголовки столбцов (в первой строке):
   - A1: `№`
   - B1: `x`
   - C1: `y`
   - D1: `$x^2+y^2$`
   - E1: `Внутри круга?`
3. Заполните столбец A номерами (например, от 1 до 1000):

- в A2 введите `1`, в A3 введите `2`, выделите обе ячейки и протяните вниз до A1001.

4. Сгенерируйте случайные координаты x и y в диапазоне от –1 до 1:
   - в B2 введите формулу: `=СЛЧИС()*2-1`
   - в C2 введите формулу: `=СЛЧИС()*2-1`
   - скопируйте эти формулы вниз до строки 1001.
5. Вычислите квадрат расстояния от центра ($x^2 + y^2$):
   - в D2 введите формулу: `=B2^2 + C2^2`
   - скопируйте вниз.
6. Определите, попала ли точка в круг (условие: $x^2+y^2 \leq 1$):
   - в E2 введите формулу: `=ЕСЛИ(D2<=1; 1; 0)`
   - скопируйте вниз.
7. Посчитайте общее число точек и число попаданий:
   - где-нибудь в свободных ячейках (например, G2 и G3) напишите:
     - G1: `Всего точек`
     - H1: `=СЧЁТ(A2:A1001)`
     - G2: `Попаданий`
     - H2: `=СУММ(E2:E1001)`
   - Вычислите приближение π:
     - G3: `Приближение π`
     - H3: `=4*H2/H1`
8. Постройте точечную диаграмму, чтобы увидеть, как точки распределились:
   - выделите диапазон B2:C1001
   - на вкладке Вставка выберите тип диаграммы Точечная (первый вариант).
   - теперь точки будут показаны на плоскости. Чтобы кружочки были маленькими, можно щёлкнуть правой кнопкой по любой точке, выбрать «Формат ряда данных», уменьшить размер маркера.
   - чтобы визуально отделить точки внутри круга от внешних, можно создать два отдельных ряда: один для точек с E2=1, другой для E2=0.

   Простой способ: отсортируйте таблицу по столбцу E, затем скопируйте координаты x и y для «внутренних» и «внешних» отдельно и добавьте их на диаграмму как новые ряды.
9. Экспериментируйте: меняйте число точек (например, сделайте 5000 или 10000) и наблюдайте, как меняется приближение π.

4. Моделирование в Python

Теперь сделаем то же самое, но с помощью кода на Python. Это удобно тем, что можно быстро менять количество точек и сразу видеть результат и график.

```
import random
import turtle

# Запрашиваем количество точек у пользователя
n = int(input("Сколько точек бросить? (например, 500): "))

# Настройка окна для рисования
screen = turtle.Screen()
screen.setup(600, 600)        # размер окна 600x600 пикселей
screen.tracer(0)              # отключаем анимацию для ускорения
turtle.speed(0)               # максимальная скорость черепашки
turtle.hideturtle()           # прячем черепашку, будем рисовать только точки

# Рисуем границу круга (окружность радиусом 200 пикселей)
```

```
turtle.penup()
turtle.goto(0, -200)          # опускаемся в нижнюю точку окружности
turtle.pendown()
turtle.color("blue")
turtle.circle(200)            # рисуем окружность радиусом 200
turtle.penup()

# Счётчик точек внутри круга
count_inside = 0

# Основной цикл: генерируем точки
for i in range(n):
    # Случайные координаты x и y в диапазоне от -1 до 1
    x = random.uniform(-1, 1)
    y = random.uniform(-1, 1)

    # Переводим координаты в экранные: умножаем на 200
    screen_x = x * 200
    screen_y = y * 200

    # Перемещаем черепашку в нужную позицию
    turtle.goto(screen_x, screen_y)

    # Проверяем, попала ли точка в круг (x² + y² <= 1)
    if x2 + y2 <= 1:
        count_inside += 1
        turtle.color("green")     # точки внутри круга - зелёные
    else:
        turtle.color("red")       # точки вне круга - красные

    # Рисуем точку размером 2 пикселя
    turtle.dot(2)

# Обновляем экран, чтобы увидеть все точки сразу
screen.update()

# Вычисляем приближение π
pi_approx = 4 * count_inside / n
print(f"Приближение числа π после {n} точек: {pi_approx}")

# Завершаем работу программы по клику на окно
turtle.done()
```

Задание для самостоятельной работы

Вычисление площади криволинейной фигуры методом Монте-Карло

Вам предстоит оценить эту площадь с помощью метода Монте-Карло, не используя интегралы.

Тема: Вычисление площади криволинейной фигуры методом Монте-Карло

Постановка задачи

Рассмотрим фигуру, ограниченную осью x, вертикальными прямыми $x = 0$ и $x = 1$ и графиком функции $y = x^2$ (парабола). Её точная площадь равна

$$S = \int_0^1 x^2\,dx = \frac{1}{3} \approx 0,33333.$$

Вам предстоит оценить эту площадь с помощью метода Монте-Карло, не используя интегралы.

Идея метода

Поместим нашу фигуру в единичный квадрат со стороной 1 (координаты x и y от 0 до 1). Площадь квадрата равна 1. Если мы будем случайным образом бросать точки в этот квадрат, то доля точек, попавших внутрь фигуры (т.е. под параболу), будет примерно равна площади фигуры:

$$S \approx \frac{\text{количество точек под параболой}}{\text{общее количество точек}}.$$

Что нужно сделать

Выполните задание в Excel или Python (на выбор).

Вариант 1: Excel

1. Создайте новую таблицу.
2. В первой строке подпишите столбцы:
   - A1: `№`
   - B1: `x`
   - C1: `y`
   - D1: `под параболой?`
3. Заполните столбец A номерами от 1 до N (например, N = 1000 или больше).
4. В ячейку B2 введите формулу случайного числа от 0 до 1: `=СЛЧИС()`.
5. В ячейку C2 введите такую же формулу: `=СЛЧИС()`.
6. В ячейку D2 введите условие проверки: `=ЕСЛИ(C2 <= B2^2; 1; 0)`.
7. Скопируйте формулы из B2, C2, D2 на все строки с точками.
8. В свободных ячейках (например, G1 и G2) посчитайте:
   - Общее число точек: `=СЧЁТ(A2:A<N+1>)`
   - Сумма попаданий: `=СУММ(D2:D<N+1>)`

- Оценка площади: `=G2/G1`

9. Дополнительно: постройте точечную диаграмму по столбцам B и C, чтобы увидеть распределение точек. Можно отдельно выделить точки, удовлетворяющие условию, и раскрасить их (например, через сортировку и добавление двух рядов).

Вариант 2: Python

Простой код (только вычисления):

```
import random

n = int(input("Введите количество точек: "))
count = 0
for _ in range(n):
    x = random.random()
    y = random.random()
    if y <= x2:
        count += 1
area = count / n
print(f"Оценка площади после {n} точек: {area}")
print(f"Точное значение: {1/3}")
```

Код с графикой (turtle):

```
import random
import turtle

n = int(input("Сколько точек бросить? "))
turtle.setup(600, 600)
turtle.speed(0)
turtle.hideturtle()
turtle.tracer(0)

# Рисуем квадрат
turtle.penup()
turtle.goto(-200, -200)
turtle.pendown()
for _ in range(4):
    turtle.forward(400)
    turtle.left(90)

# Рисуем параболу y = x^2, но x и y надо масштабировать: от 0 до 1 -> от -200 до 200
turtle.penup()
turtle.goto(-200, -200)  # начало координат в левом нижнем углу
turtle.pendown()
turtle.color("blue")
for x in range(-200, 201, 5):
```

```
    # переводим экранные координаты в математические: x_math = (x_scr + 200)/400
    x_math = (x + 200) / 400
    y_math = x_math2
    y_scr = -200 + y_math * 400
    turtle.goto(x, y_scr)
turtle.penup()

count = 0
for i in range(n):
    x_math = random.random()
    y_math = random.random()
    x_scr = -200 + x_math * 400
    y_scr = -200 + y_math * 400
    turtle.goto(x_scr, y_scr)
    if y_math <= x_math2:
        count += 1
        turtle.color("green")
    else:
        turtle.color("red")
    turtle.dot(2)

turtle.update()
print(f"Оценка площади: {count/n}")
turtle.done()
```

Исследование

Проведите эксперименты с разным количеством точек: 100, 1000,10000,100000. Занесите результаты в таблицу и сделайте вывод:

- Как меняется оценка площади при увеличении n?
- Насколько близко полученное значение к реальному значению интеграла?
- Попробуйте посчитать другие виды интегралов?

Прогноз результатов баскетбольного матча

Представьте, что завтра играют две команды. Мы знаем, что Лидер забивает в среднем 60% бросков с игры, а Новичок — только 40%. В матче каждая команда сделает по 20 бросков. Можно ли точно сказать, сколько очков наберет каждая команда? Нет, потому что каждый бросок — случайное событие.

Идея метода Монте-Карло здесь: Мы можем смоделировать этот матч 10 000 раз. В каждой симуляции мы "бросаем" 20 раз для каждой команды, используя их вероятности попадания. Затем мы смотрим, в скольких процентах случаев победил Лидер.

Моделирование в Excel: Баскетбольный матч

Мы создадим таблицу, которая симулирует одну игру, а затем с помощью множества таких симуляций оценит шансы на победу.

Пошаговая инструкция

1. Настройка параметров:
   * В ячейке A1 напишите `Вероятность Лидера`.
   * В ячейке B1 введите `0,6` (60%).
   * В ячейке A2 напишите `Вероятность Соперника`.
   * В ячейке B2 введите `0,4` (40%).
   * В ячейке A3 напишите `Бросков за матч`.
   * В ячейке B3 введите `20`.

2. Создаем таблицу для одной симуляции (одного матча):
   * В ячейках A5:A24 проставьте номера бросков от 1 до 20.
   * Столбец B (Бросок Лидера): В ячейку B5 введите формулу: `=ЕСЛИ(СЛЧИС() <= $B$1; 1; 0)`. Эта формула генерирует случайное число от 0 до 1. Если оно меньше или равно мастерству игрока (0.6), бросок засчитывается (1), иначе промах (0). Скопируйте формулу вниз до B24.
   * Столбец C (Бросок Соперника): В ячейку C5 введите формулу: `=ЕСЛИ(СЛЧИС() <= $B$2; 1; 0)`. Скопируйте её вниз до C24.

3. Подводим итоги одного матча:
   * В ячейке D5 напишите `Очки Лидера`.
   * В ячейке E5 введите формулу подсчета суммы попаданий: `=СУММ(B5:B24)`.
   * В ячейке D6 напишите `Очки Соперника`.
   * В ячейке E6 введите: `=СУММ(C5:C24)`.
   * В ячейке D7 напишите `Победитель`. В ячейке E7 введите формулу, определяющую победителя: `=ЕСЛИ(E5 > E6; "Лидер"; ЕСЛИ(E5 < E6; "Соперник"; "Ничья"))`.

Сейчас вы нажатием клавиши F9 (пересчет листа) можете видеть, как меняется результат матча. Иногда выигрывает сильный игрок, иногда слабый (редко), иногда бывает ничья.

4. Множество симуляций (Монте-Карло!):

Теперь нам нужно провести этот эксперимент много раз (например, 1000 раз) и посчитать статистику.

   * В столбце H (начиная с H2 и вниз до H1001) проставьте номера симуляций от 1 до 1000.
   * Нам нужно создать механизм, который будет пересчитывать наш матч 1000 раз и записывать результаты. К сожалению, стандартные функции Excel не могут просто так взять и повторить вычисления из E7. Здесь нам понадобится Таблица данных (Data Table) — мощный инструмент "что-если" анализа.
   * В ячейку I1 напишите заголовок `Результат`.
   * В ячейку I2 введите ссылку на результат нашего матча: `=E7`. (Это свяжет таблицу данных с моделью).
   * Выделите диапазон H1:I1001 (включая заголовки).
   * Перейдите на вкладку Данные -> Анализ "что-если" -> Таблица данных.
   * В появившемся окне в поле "Подставлять значения по столбцам" укажите любую пустую ячейку, например G1. (Excel будет подставлять разные значения из столбца H в ячейку G1, каждый раз пересчитывая лист и записывая результат из I2 в столбец I). Нажмите OK.

Excel мгновенно заполнит 1000 строк в столбце I результатами "Лидер", "Соперник" или "Ничья".

5. Анализ результатов:
   * В ячейке K1 напишите `Побед Лидера`.
   * В ячейке K2 введите формулу: `=СЧЁТЕСЛИ(I2:I1001; "Лидер")`.
   * В ячейке L1 напишите `Всего игр`.
   * В ячейке L2 введите: `=СЧЁТ(I2:I1001)`.
   * В ячейке M1 напишите `Шанс на победу`.
   * В ячейке M2 введите формулу: `=K2/L2`. Отформатируйте ячейку как процентный формат.

Результат: Вы получите объективную оценку. Даже если игрок бьет с 60% точности, он не выигрывает 100% матчей. Монте-Карло показал, что его шанс на победу составляет, например, 88%, а 12% матчей он может проиграть из-за случайности (или сыграть вничью).

Моделирование в Python: Баскетбольный турнир

В основной функции задаются параметры, запускается 10 000 симуляций, подсчитываются победы и выводится распределение разницы счета.

```
import random

# Параметры симуляции
prob_leader = 0.6         # вероятность попадания Лидера (60%)
prob_rookie = 0.4         # вероятность попадания Новичка (40%)
shots_per_game = 20       # бросков за матч
num_simulations = 10000   # количество матчей

print(f"Симуляция {num_simulations} баскетбольных матчей...")
print(f"Лидер забивает {prob_leader*100:.0f}% бросков")
print(f"Новичок забивает {prob_rookie*100:.0f}% бросков")
print(f"За матч каждая команда делает {shots_per_game} бросков\n")

# Список для хранения разницы очков (Лидер - Новичок) в каждом матче
differences = []

# Счётчики исходов
wins_leader = 0
wins_rookie = 0
draws = 0

# Основной цикл по всем матчам
for _ in range(num_simulations):
    # Симулируем броски Лидера
    leader_score = 0
    for _ in range(shots_per_game):
        if random.random() < prob_leader:
            leader_score += 1

    # Симулируем броски Новичка
```

```
    rookie_score = 0
    for _ in range(shots_per_game):
        if random.random() < prob_rookie:
            rookie_score += 1

    # Вычисляем разницу
    diff = leader_score - rookie_score
    differences.append(diff)

    # Подсчитываем исход матча
    if diff > 0:
        wins_leader += 1
    elif diff < 0:
        wins_rookie += 1
    else:
        draws += 1

# Вывод итоговой статистики
print("РЕЗУЛЬТАТЫ:")
print(f"Побед Лидера:   {wins_leader} ({wins_leader/num_simulations*100:.2f}%)")
print(f"Побед Новичка:  {wins_rookie} ({wins_rookie/num_simulations*100:.2f}%)")
print(f"Ничьих:         {draws} ({draws/num_simulations*100:.2f}%)")
```

 # Можно построить текстовую гистограмму

```
# Задаём интервалы от -20 до 20 с шагом 2
bins = list(range(-20, 21, 2))  # [-20, -18, -16, ..., 18, 20]
counts = [0] * len(bins)        # массив для подсчёта попаданий в каждый интервал

# Распределяем все значения разницы по интервалам
for diff in differences:
    for i, bound in enumerate(bins):
        if diff <= bound:
            counts[i] += 1
            break

# Печать гистограммы
print("\nГистограмма разницы очков (Лидер - Новичок):")
print("-" * 50)
max_count = max(counts)
scale = 50 / max_count if max_count > 0 else 1   # масштабируем длину до 50 символов

for i, bound in enumerate(bins):
    # Определяем левую границу интервала
    left = bins[i-1] if i > 0 else "менее"
    bar_length = int(counts[i] * scale)
    print(f"{left}...{bound:3d}: {'' * bar_length} ({counts[i]})")
```